THE META ARCHITECT — AI RELIABILITY ENGINEERING

The STATE Field Guide

A field reference for engineers who inherited a GenAI platform, got handed the pager, and need an architecture — not a better prompt.

S	T	A	T	E
STRUCTURED	TRACEABLE	AUDITABLE	TOLERANT	EXPLICIT

State beats intelligence. *A mid-tier model with proper state management outperforms a frontier model running stateless — every time.*

Simon Paris
I design AI systems that don't break.

FIELD GUIDE — EDITION 01
SIMONPARIS.CA

HOW TO USE THIS GUIDE

You didn't choose this.

Someone handed you a GenAI platform, called it production-ready, and moved on to the next initiative. Now it's non-deterministic, undebuggable, and risk is asking questions you can't answer. This guide doesn't teach you a better model. It shows you the five properties your system is missing — and exactly what breaks in their absence.

EVERY PILLAR PAGE FOLLOWS ONE STRUCTURE

01 **Definition** — what the pillar means, in one sentence.

02 **Failure mode** — what happens in production when it's missing.

03 **Diagnostic question** — ask it about your system right now.

04 **Field evidence** — the number or principle behind it.

CONTENTS

THE THESIS

State beats intelligence.

A mid-tier model with proper state management outperforms a frontier model running stateless — every time. Production AI systems don't fail because the model is too weak. They fail because nobody gave the system a memory, a witness, or a way to say no.

STATE governs every pipeline operation. Risk tier determines which pillars are mandatory — not preference, not budget, not how confident the demo looked.

Risk Tiers

TIER	CONDITION	REQUIRED PILLARS
Low	Internal, read-only, no API calls, non-personalized	S + T
Medium	Database writes, LLM calls, external API calls, external-facing	S + T + E
High	Decisions affecting individuals, financial operations, regulated data, Law 25 scope	All five — no exceptions

S+T+E is the floor, not a target. Every Meta Architect pipeline command — every database write, every LLM call, every external API call — runs at **Medium risk minimum**. There is no tier below it that skips a pillar.

The diagnostic habit this guide builds: for every pillar, two questions decide whether you're covered — a state question and an evidence question. If you can't answer both without opening a conversation log or asking someone on the team to "check," the pillar isn't there yet. It's a hope.

IN PRACTICE

Two teams ship the same RAG pipeline against the same model. Team A wires it up with no state object, no lock pattern, no validation gate — it works in the demo, so it ships. Team B wraps a smaller, cheaper model in explicit state, full call logging, and a validation gate before every write. Six weeks in, both systems are hitting the same rate of model-level mistakes — that part was never the differentiator. Team A is debugging by re-reading Slack threads and re-running the pipeline by hand, hoping to catch the bug live. Team B is filtering a trace table by `stage` and `status`, and has a fix out before the standup. Same model. Same failure rate. Different architecture underneath it. That's the thesis, operationalized — not a claim about model quality, a claim about what surrounds it.

S Structured

Explicit state schemas, not implicit context.

Every operation initializes a typed state object before doing any work. Business entities — claims, tickets, portfolios, content items — are first-class state objects with defined schemas. The `stage` field always reflects current execution position, not what the conversation history implies it should be.

THE FAILURE MODE IT PREVENTS

Context rot. The agent loses track of where it is in a workflow as the context window fills. Older state gets silently dropped. Behavior degrades — and nothing throws an error to tell you why.

> DIAGNOSTIC QUESTION
>
> "If this agent crashed right now, could you look at the last saved state object and know exactly where in the workflow it stopped – without reading the conversation history?"

73.5% faster execution, **60% fewer LLM calls** — CA-MCP research, measuring the move from implicit context to shared explicit state.

FIELD APPLICATION

- A workflow's **stage** field is the single source of truth for "where are we" — never inferred from the last few messages in a transcript.
- Business entities — a ticket, a claim, a draft — are rows with schemas, not free-floating conversation context.

"This stuff is non-deterministic."

HOW PRACTITIONERS ALREADY DESCRIBE THIS

S T A T E

T Traceable

Every step observable, every decision logged.

Log every LLM call, external API call, and meaningful stage transition — with every required field, every time. No silent operations. You must be able to reconstruct exactly what the agent did, what it was given, and what it produced, for any execution, after the fact, without guessing.

THE FAILURE MODE IT PREVENTS

Blind debugging. "The model did something weird" as a post-mortem. Failures that only surface through downstream effects, three systems away from where they started.

> DIAGNOSTIC QUESTION
>
> `"Can you pull up a trace right now showing every LLM call, tool call, input, and output for a specific execution from last week?"`

1 in 3 enterprise teams are satisfied with their AI observability and evaluation — Cleanlab, 2025. The other two are debugging by vibes.

FIELD APPLICATION

- Every LLM call gets a log row with model version, output summary, and status — written at call time, not reconstructed later from memory.
- "What did the agent see" and "what did it do" are two different fields. Both mandatory, both queryable.

"There's no stack trace."

HOW PRACTITIONERS ALREADY DESCRIBE THIS

A Auditable

Governance-ready, explainable under Law 25 / OSFI / EU AI Act.

For any automated decision affecting an individual or using personal data, write a decision record: what personal data was used, what the principal factors were, what model and prompt version ran, and who can review it.

THE FAILURE MODE IT PREVENTS

Regulatory exposure. The inability to answer a regulator's — or an individual's — question about why an automated decision was made. Not a technical gap. A liability.

> DIAGNOSTIC QUESTION
>
> "If a regulator sent a request today asking what data your system used to make a specific decision last month, could you answer it inside 30 minutes?"

$25M or 4% of global revenue — Quebec Law 25's penal maximum for failing to disclose an automated decision on request ($10M / 2% administrative). Applies broadly, not just to legal-status decisions.

Not required for low-risk internal tooling with no personal data. Required the moment a real person's data enters the pipeline.

FIELD APPLICATION

- A decision record answers three questions before anyone has to ask them: what data, what factors, what model.
- "We didn't think to log that" is not a defensible answer to a Law 25 access request — a PIA is mandated before AI touches personal data at all.

"Can we log why the agent did this?"

HOW PRACTITIONERS ALREADY DESCRIBE THIS

S T **A** T E

T Tolerant

Fault-tolerant and resumable after failure.

When the workflow fails at step 6, it resumes from step 6 — not step 1. Lock before expensive operations. Clear the lock on failure. Every command is retryable. Failed runs leave no permanent broken state behind.

THE FAILURE MODE IT PREVENTS

Full restarts after partial failures. Lost work. Systems that only work in the happy-path forward motion — and fall over the first time reality doesn't cooperate.

> DIAGNOSTIC QUESTION
>
> "If this workflow crashes at step 6 of 10 right now – does it resume from step 6, or restart from step 1?"

70% of **regulated** enterprises replace at least part of their AI stack every three months — 41% of unregulated ones too. Cleanlab, 2025. See page 15.

FIELD APPLICATION

- A lock timestamp is set before the expensive operation starts, not logged after — so a crash mid-operation is visible instead of silent.
- Retry means resume, not rerun. A workflow that reruns step 1 after failing at step 6 duplicates the work and the cost.

"A clever prototype duct-taped into production."

HOW PRACTITIONERS ALREADY DESCRIBE THIS

E Explicit

Deterministic boundaries, no magic.

Every LLM or external API output passes through a validation gate before any write or action. Invalid output routes to the error path — never a silent continue. Every point where reasoning turns into a real-world action is a named, validated gate.

THE FAILURE MODE IT PREVENTS

Hallucination cascades. The model wraps JSON in a markdown code fence, the parser dies without a sound. A confident, wrong answer triggers a downstream action with no contract check to stop it.

> DIAGNOSTIC QUESTION
>
> "For every LLM call in this workflow — what's the worst thing it could output, and what actually stops that from becoming a real-world action?"

Seams vs. Components

Testing individual nodes isn't the same as testing the workflow. Always run one full end-to-end execution with live LLM output before shipping — pinned data proves the nodes, not the seams between them.

FIELD APPLICATION

- A validation gate has a name and a failure path. "Parse the JSON" isn't a gate — "parse the JSON or route to error_invalid_output" is.
- The worst-case output of every LLM call is a design input, not an edge case you'll get to later.

"I just want it to say 'I don't know' instead of hallucinating."

HOW PRACTITIONERS ALREADY DESCRIBE THIS

S T A T E

SCORING

Score it. Don't guess it.

Each pillar scores 0–2, based on its two diagnostic questions. Both yes: 2. One yes: 1. Both no, or "not sure": 0.

SCORE	LABEL
0–3	Critical Risk
4–5	High Risk
6–7	Developing
8–9	Production-Ready
10	STATE-Compliant

Where existing frameworks stop

Orchestration and observability tools, evaluated against the same five reliability properties.

FRAMEWORK	S	T	A	TOL	E
LangGraph	◧	◧	☐	■	◧
W&B Weave	☐	■	☐	☐	◧
LangSmith	☐	■	☐	☐	☐
Arize Phoenix	☐	■	◧	☐	☐
OpenTelemetry GenAI	☐	■	☐	☐	☐

■ Strong ◧ Partial ☐ Absent

A is the weakest-covered pillar here — absent outright in four of five, and even Arize Phoenix's partial credit is prompt/model versioning plus manual annotations, not a native decision-record artifact. That's not a marketing claim about STATE. That's a gap in the tools you're already using.

REFERENCE CARD

Schemas & Patterns

STATE OBJECT SCHEMA

```
{
  workflowId: string,    // crypto.randomUUID() per run
  stage: string,         // current stage name
  entityType: string,    // "idea" | "post" | "hook"
  entityId: string,      // Supabase row id (uuid)
  startedAt: string,     // ISO timestamp
  lastUpdatedAt: string // ISO, updated per stage
}
```

LOG ENTRY SCHEMA

```
{
  workflow_id, entity_id,
  step_name: string,    // e.g. "research_architect"
  stage: string,        // stage at time of log
  timestamp: string,
  output_summary: string,
  model_version: string, // actual model, never
                        // hardcoded in a template
  status: "success" | "error"
}
```

LOCK PATTERN

```
// 1. Set BEFORE the operation
updateRecord(TABLE, id, {
  research_started_at: now()
});

// 2. Clear on failure
updateRecord(TABLE, id, {
  research_started_at: null,
  status: "research_failed"
});
```

IDEMPOTENCY CHECK

```
if (row.research_started_at
    !== null) {
  return "Already in progress —
    check status before retry.";
}
```

ERROR FORMAT

```
❌ [Command] failed at [stage]
  — [error message]
  — lock reset, safe to retry
```

CHECKLIST — MEDIUM RISK MINIMUM

The S+T+E Checklist

Database writes, LLM calls, external APIs, anything external-facing. This is the floor for a content pipeline, not the ceiling.

- ☐ State object initialized with all required fields
- ☐ Stage updated at each transition
- ☐ Every LLM call logged to the **logs** table with required fields
- ☐ Every external API call logged to the **logs** table
- ☐ Lock set before expensive operations
- ☐ Lock cleared on failure
- ☐ All LLM/API output validated before any database write
- ☐ Error path reports: stage + error message + confirms lock reset

A single unchecked box means the system is not production-ready — regardless of what the model scored on a benchmark.

Score it right here

Run S, T, and E through the two diagnostic questions on their pillar pages (04–08). Both yes: 2. One yes: 1. Neither, or "not sure": 0. **0–5 across the three** means this checklist is aspirational, not descriptive. Full rubric and risk bands: page 09.

WHERE TEAMS USUALLY FAIL THIS CHECKLIST FIRST

→ The lock is set, but never cleared on failure — so a crashed run looks "still in progress" forever.

→ LLM output gets validated for shape, not content — the JSON parses, but nobody checked it against the actual write contract.

CHECKLIST — HIGH RISK / REGULATED DATA

The All-Five Checklist

Decisions affecting individuals. Financial operations. Regulated data. Law 25 scope. Everything on page 11, plus:

- ☐ Decision record written for any decision affecting an individual
- ☐ Personal data fields enumerated in the decision record
- ☐ Model version and prompt version captured in the decision record
- ☐ Human-in-the-loop approval node present for high-risk decisions
- ☐ Retention policy documented for decision records

All Five — no exceptions. This is the tier where a missing pillar isn't a technical debt item. It's the line a regulator draws around your exposure.

BEFORE YOU GET TO THE CHECKLIST

Quebec Law 25 mandates a Privacy Impact Assessment before AI touches personal data at all — this checklist is what the PIA is checking for, not a replacement for it. If nobody on the team can point to the PIA, that's item zero, above the five.

Reference: page 06 (Auditable) for the decision-record schema this checklist assumes; page 09 for how the Auditable pillar scores against tools you may already be using.

THE FAILURE TAXONOMY

Ten failures. Five root causes.

Every one of these has been the subject of a 2am page, an incident review, or a "the model did something weird" post-mortem. None of them are model problems.

FAILURE MODE	WHAT YOU'LL SEE	PILLAR	THE FIX
Context Rot	Agent's behavior drifts over a long session; no error, just wrong.	S	Typed state object per operation; *stage* always reflects real position.
"The Model Did Something Weird"	Same input, different output; the bug can't be reproduced.	S	State object logged at every transition — the model isn't the post-mortem, the state is.
Blind Debugging	No trace of a specific execution from last week; nothing to pull up.	T	Every LLM/API call logged with full required fields, no exceptions.
Flying Blind	No metrics on hallucination or drift; problems surface via user complaints.	T	Full trace reconstruction for prompt chains and tool calls, not just prompts.
Hallucination Cascade	JSON wrapped in a markdown fence; parser dies silently, action fires anyway.	E	Named validation gate before every write — invalid output routes to error, never continues.
Prompt Whack-a-Mole	Every prompt fix breaks something else; accuracy can't hold.	E	Seams vs. Components — full end-to-end test with live output, not just the node.
The Regulatory Blind Spot	"Can we log why the agent did this?" — and the answer is no.	A	Decision record per automated decision: data used, principal factors, model/prompt version.
Full Restart	Crash at step 6 of 10; workflow reruns from step 1, work is lost.	Tol	Lock before expensive operations, resume from *stage*, not from zero.
The 90-Day Rebuild	The agent system gets rebuilt from scratch, again, on a predictable cycle.	Tol	The Reboot Test, passed before "production-ready" gets said out loud.
The Vibes-Based Go/No-Go	A pilot ships to production because the demo looked good.	All 5	Score it with the STATE rubric before promotion — not after the incident.

FIELD NOTES

Three failures worth slowing down for.

The Regulatory Blind Spot

Risk and legal are already asking: "Can we log why the agent did this? What data did it use?" Quebec's Law 25 doesn't treat this as optional — it requires disclosure of the personal data used, the principal factors behind an automated decision, and a path for the individual to request human review. Miss it and the exposure isn't hypothetical: up to $25M or 4% of global revenue, penal. Ask yourself: could you answer a regulator's request about last month's decision inside 30 minutes?

The 90-Day Rebuild

Cleanlab's 2025 research found that 70% of regulated enterprises — and 41% of unregulated ones — replace at least part of their AI stack every three months. Not always the whole system, but enough of it, often enough, that continuity never holds. That's what happens to a system that only survives the happy path: the first crash, timeout, or dependency outage that wasn't in the demo forces a teardown instead of a resume. The Reboot Test is one way to catch this before the next churn cycle does — restart the service, reboot the container, inject a failure mid-workflow, take a dependency offline for two minutes. If it only works moving forward, it's a demo, not a system.

The Vibes-Based Go/No-Go

Every data leader feels pressure to ship GenAI; almost none of them think the timelines are realistic. Between those two facts sits a go/no-go decision made on how good the demo looked, not on a score. The STATE rubric exists so that decision has a number attached to it — 0 to 10, five pillars, two diagnostic questions each. A system that scores 4 and ships anyway isn't a bet. It's a known risk nobody wrote down.

Hallucination Cascade

The failure isn't the hallucination — models will always occasionally be wrong. The failure is what happens next: JSON wrapped in a markdown fence, a parser that dies without a sound, a confident wrong answer that fires a downstream action because nothing checked it first. "I just want it to say 'I don't know' instead of hallucinating" is the most common way practitioners put this — and it's a validation-gate problem, not a prompting problem. For the one LLM call in your workflow that writes to something real: what actually stops the worst output from becoming a real action?

NAMED PRINCIPLES

Three ideas worth naming.

The Reboot Test

Before declaring a system production-ready, simulate the state transition:

- → Restart the service
- → Reboot the container
- → Inject a failure at an intermediate step
- → Take an external dependency offline for two minutes

If the system only works moving forward, it's a demo. If it survives state transitions, it's closer to real.

Seams vs. Components

Testing individual nodes is component testing. Testing the full workflow with live LLM output flowing through every boundary is integration testing. Both are required. Pinned data proves the nodes work. Only a live run proves the seams do too.

The STATE Question

"Does your agent pass the STATE test?" Applies to any AI system, any stack, any scale.

What STATE is not

- – Not a model evaluation framework. Doesn't assess model quality or capability.
- – Not an MLOps framework. Doesn't cover training pipelines or dataset management.
- – Not a security framework. Doesn't replace threat modeling.
- – Deliberately incomplete. Covers the five properties most commonly missing in production agent systems.

“I design AI systems that don’t break.”

— SIMON PARIS, THE META ARCHITECT

The Meta Architect is Simon Paris's practice in AI Reliability Engineering: production-grade agent systems for data-sensitive enterprises navigating Law 25, OSFI, and the EU AI Act. Not prompt engineering. Not AI hype. The plumbing underneath both.

S	T	A	T	E
STRUCTURED	TRACEABLE	AUDITABLE	TOLERANT	EXPLICIT